

Supervised Fine-Tuning

From text predictor to assistant · video 2:08

This lesson answers one question: how does a text predictor become an assistant? Asked questions in chat format, the base Qwen2.5-0.5B never answers in the right form and never stops, although with a plain prompt it knows 75% of the answers. **Supervised fine-tuning** on 280 example conversations, with the loss only on the assistant's tokens, takes it to 95% exact answers on held-out questions, and it stops every time. Its three mistakes show what fine-tuning does not add: knowledge and skills.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. What the base model lacks	▶ 0:08-0:42	1	☐
2. Data and the loss mask	▶ 0:42-1:24	3	☐
3. Results, and what SFT doesn't add	▶ 1:24-1:59	1	☐

Before you start: Deep Dive episodes 17 (cross-entropy) and 19 (Adam/AdamW); LLMs in Practice episodes 1 (prompts are just context) and 9 (fine-tuning and LoRA). Code: `code/d36_sft` (Qwen2.5-0.5B, about 8 GB of RAM to train).

CONCEPT 1 OF 3

What the base model lacks

▶ Video 0:08-0:42

A base model only continues text. Asked 60 questions in the chat format (special tokens marking user and assistant), the base Qwen2.5-0.5B gives **0%** answers in the right form and **never stops** within 40 tokens: it repeats the question and emits junk tokens. With a plain prompt, “Question: What is 63 + 34?\nAnswer:”, the same model has the right answer in its first line **75%** of the time (40% in exactly the expected form).

Qwen2.5-0.5B base, 60 questions

chat format	plain prompt
<pre>< im_start >user What is 63 + 34?< im_end > < im_start >assistant → "What is 63 + 34? <junk> What is 63 + 34? <junk> ..." right form 0% · stops 0%</pre>	<pre>Question: What is 63 + 34? Answer: → "63 + 34 = 97" right answer in 75%</pre>

the knowledge is there; the behaviour is missing

▶ 0:41 60 questions in chat format: right form 0%, stops 0%. With a plain “Question: ... Answer:” prompt: the right answer in 75%.

KEY IDEA

The knowledge is learned in pre-training; what's missing is the behaviour: answer in the expected form, then stop.

Check 1: What the base model lacks

Answer all 1 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“The base model fails in chat format mainly because it does not know the capitals.”

True ☐ False ☐

CONCEPT 2 OF 3

Data and the loss mask

▶ Video 0:42-1:24

The training data are examples of the wanted behaviour: 280 short conversations (100 additions, 30 capitals × 3, 30 words in capitals × 3) in the chat template: `<|im_start|>user ... <|im_end|>`
`<|im_start|>assistant ... <|im_end|>`.

280 example conversations

What is 63 + 34? → 63 + 34 = 97.
What is the capital of Peru? → The capital of Peru is Lima.
Write the word 'needle' in capital letters. → NEEDLE

```
<|im_start|>user
What is 63 + 34?
<|im_end|>
<|im_start|>assistant
63 + 34 = 97.
<|im_end|>
```

▶ 0:56 280 conversations of 3 checkable tasks, in Qwen's chat template.

next-token prediction, loss only on the answer

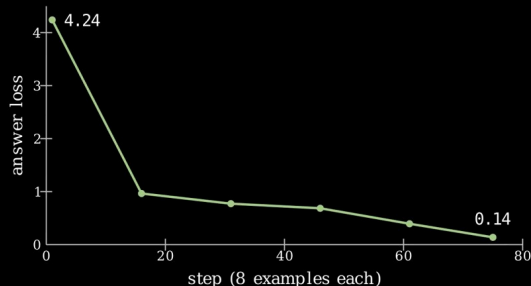
```
<|im_start|>user \n What \n is \n 6 3 \n + \n 3 4 \n ? <|im_end|> \n <|im_start|>assistant \n
prompt: no loss (label -100)
```

```
6 3 \n + \n 3 4 \n = \n 9 7 \n . <|im_end|>
answer: loss
```

this example: 12 of 30 tokens - a training batch: 31%

▶ 1:13 Prompt tokens get label -100 (no loss); the answer and its end token are trained: 12 of 30 tokens here, 31% of a batch.

Training is ordinary next-token prediction with one twist: the loss counts only the assistant's tokens. Every other label is set to -100, which cross-entropy ignores. In one example that leaves 12 of 30 tokens; in a batch, 31%. The model is not taught to write questions, only to answer them, and because the end token belongs to the answer, it learns to stop. Full fine-tuning, 75 steps of 8 examples, took the answer loss from 4.24 to 0.14 in a few minutes on a CPU.



▶ 1:22 Answer loss over 75 steps of 8: 4.24 → 0.14.

KEY IDEA

SFT = next-token prediction on curated conversations, with the loss masked to the assistant's part, end token included.

Check 2: Data and the loss mask

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

A conversation has 18 prompt tokens and a 12-token answer (end token included). What share of the tokens contribute to the loss?

2.2 SHORT ANSWER

What would go wrong if the end token were left out of the trained labels?

2.3 HANDS-ON CODE

Try it yourself. Build masked labels and check which positions count.

```
import torch, torch.nn.functional as F
prompt, answer = [5, 6, 7, 8], [9, 10, 2]
labels = torch.tensor([-100] * len(prompt) + answer)
logits = torch.randn(len(labels), 12)
print(F.cross_entropy(logits, labels, ignore_index=-100))
print((labels != -100).sum())
```

CONCEPT 3 OF 3

Results, and what SFT doesn't add

▶ Video 1:24-1:59

On 60 held-out questions, with numbers, countries and words never seen in training: **95%** exact answers, stopping **every time** after 7 tokens on average. The three misses: “48 + 45 = 113”, “The capital of Ukraine is Kiev” (the older spelling, so the expected answer “Kyiv” is missed), and “flower” in capitals becoming “FLORAL”.

60 held-out questions (new numbers, countries, words)

	base	fine-tuned
exact answer	0%	95%
stops by itself	0%	100%
mean length	40 tokens	7 tokens

▶ 1:35 60 held-out questions: exact answers 0% → 95%; stops 0% → 100%; length 40 → 7 tokens.

the 3 misses

```
What is 48 + 45?  
→ 48 + 45 = 113.  
  
What is the capital of Ukraine?  
→ The capital of Ukraine is Kiev.  
  
Write the word 'flower' in capital letters.  
→ FLORAL
```

fine-tuning taught the format; knowledge and skills come from pre-training

▶ 1:49 The 3 misses: 48 + 45 = 113, “Kiev”, FLOWER → “FLORAL”.

KEY IDEA

Fine-tuning teaches format and behaviour quickly; facts and skills still come from pre-training, and its errors show through.

Check 3: Results, and what SFT doesn't add

Answer all 1 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

The held-out countries never appeared in the fine-tuning data, yet most answers are right. Why?

- A. The test leaked into training
- B. The model already knew them from pre-training; SFT taught it how to answer
- C. Fine-tuning taught it geography
- D. It guessed

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. What the base model lacks · rewatch ▶ 0:08-0:42

1.1 False. With a plain prompt it gets 75% right; the chat tokens are simply unfamiliar territory for it.

2. Data and the loss mask · rewatch ▶ 0:42-1:24

2.1 40%. 12 / 30.

2.2 The model would never learn to emit it, so it would keep generating after the answer. Stopping is a learned behaviour like any other token.

2.3 A loss averaged over 3 positions. Positions labelled -100 are skipped in both the sum and the average.

3. Results, and what SFT doesn't add · rewatch ▶ 1:24-1:59

3.1 B. The plain-prompt test showed the knowledge was there before fine-tuning.